Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0309 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE ENGINEERING

Submitted by

PUJITHA K (192524149)
SATHWIKA REDDY A (192525274)
ROHINI SREE R (192524471)

Under the Supervision of

Dr.Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department	AI&DS
Programme	Computer Science Engineering
Course Code & Course Name	CSA0309 – Data Structures
Academic Year / Batch	2026-2027
Faculty Name	Dr. Uma Priyadarsini P.S
Assignment Title	Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the high-speed security log processing system. CO3: Make use of self-balancing trees (AVL/Red-Black Trees) for ordered, dynamically balanced storage of risk-ranked events. CO4: Make use of hashing, collision-resolution techniques, and priority queues for duplicate detection and high-risk event ranking in C. CO5: Develop a robust real-time cyber threat detection engine and evaluate its search performance under varying load factors and data distributions.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate / Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure; SDG 16 – Peace, Justice and Strong Institutions
Industry / Societal Relevance	Real-time cybersecurity monitoring, duplicate-IP detection, risk prioritization, efficient security-log processing, and scalable threat analysis.

B. Assignment Problem / Challenge

The assignment requires the development of a realistic real-time cyber threat detection engine that applies data-structure concepts to high-volume security event processing. The solution must identify repeated IP addresses quickly, rank suspicious events by risk, maintain ordered risk information using a self-balancing tree, and surface the highest-risk events using a priority queue.

The design must also consider scalability, collision handling, load factors, and skewed distributions. The implementation should demonstrate appropriate engineering decisions and validate the design through testing and performance comparison.

C. Problem Statement

A cybersecurity operations center is developing a real-time threat-detection engine that ingests security event logs such as login attempts, IP connections, and access requests at high volume. Each event contains a source IP, timestamp, event type, and risk indicators. The system must process a very large stream with minimal latency, identify duplicate or repeated IP addresses, rank suspicious and high-risk events ahead of routine events, remain efficient as the number of distinct IPs and events grows, and handle varying load factors and skewed data distributions.

The proposed C program uses a hash table with collision resolution for average O(1) duplicate-IP and event lookup, an AVL tree for dynamically sorted risk-ranked storage and ordered reporting, and a priority queue for continuously surfacing the highest-risk events. Performance is evaluated under different load factors and key distributions.

D. Requirements and Constraints

- Functional requirements: store security events, detect repeated source IPs, rank events by risk, provide ordered reporting, and retrieve the highest-risk events.
- Performance requirements: average-case constant-time hashing operations and efficient balanced-tree and priority-queue operations.
- Technical constraints: implementation must be in C using appropriate structures, functions, memory management, and collision handling.
- Scalability constraint: the design should remain practical for large event streams.
- Reliability constraint: duplicate detection and risk ranking must remain correct under collisions and repeated inputs.
- Security/privacy consideration: the program uses sample IP addresses and does not require real personal data.
- Validation requirement: test multiple load factors and both distributed and skewed IP distributions.
- Documentation requirement: include pseudocode, implementation, results, analysis, reflection, and references.

E. Student Work

1. Problem Understanding and Formulation

The problem is to design a real-time data-processing system for security events where fast lookup, duplicate detection, ordered risk storage, and highest-risk retrieval are all required simultaneously. A

single data structure is not ideal for every operation, so the solution combines hashing, an AVL tree, and a max-heap priority queue.

The available information for each event is a source IP address, timestamp, event type, and risk score. The implementation assumes that risk is represented by an integer from 0 to 100 and that source IP strings are valid IPv4-style addresses. The sample program uses a fixed demonstration dataset while also containing functions that can generate larger synthetic datasets for performance testing.

The major constraints are processing speed, collision handling, memory use, correctness under repeated IPs, and maintaining ordered risk information as events arrive.

2. Application of Course Knowledge

Hashing is used because it provides average O(1) insertion and lookup when the hash function distributes keys effectively. Separate chaining is selected for collision resolution because multiple IP addresses can safely occupy the same bucket through linked lists.

An AVL tree is used to maintain events in sorted order by risk score and a unique event identifier. AVL rotations keep the tree height O(log n), making ordered insertion and search efficient.

A max-heap priority queue is used to retrieve the event with the highest risk score efficiently. Insertion takes O(log n) and retrieving the maximum takes O(1), with removal taking O(log n).

For a hash table with n stored keys and m buckets, the load factor is $\alpha = n/m$. Increasing α generally increases the expected chain length and therefore can increase lookup time. This relationship is examined using different table sizes and synthetic distributions.

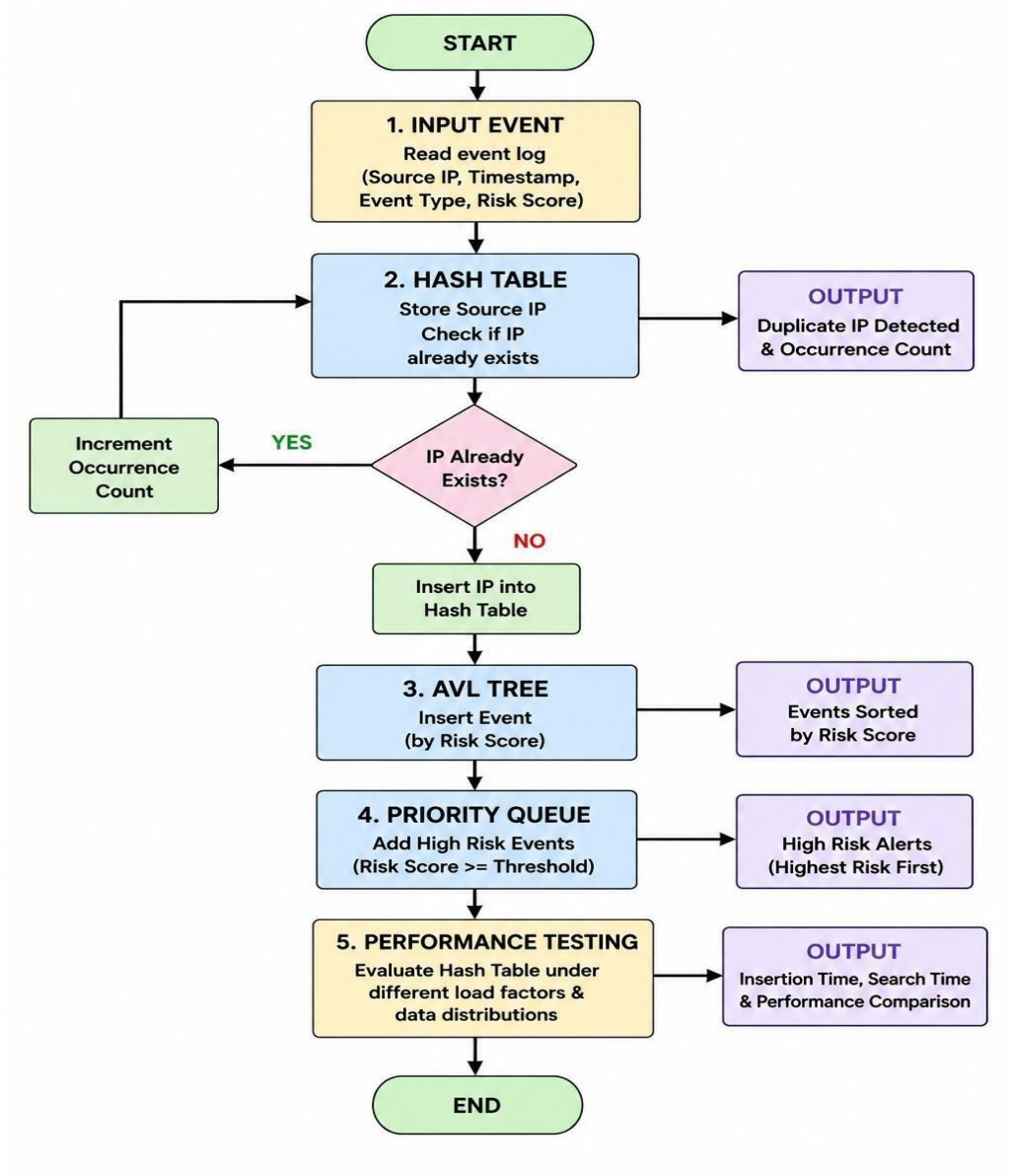
3. Solution / Design / Methodology

System Architecture

The input event first enters the hash table. The source IP is hashed to a bucket. If the IP already exists, its occurrence count is incremented and the event is marked as a repeated-IP event. Every event is inserted into the AVL tree using risk score and event ID as the ordering key, and is also inserted into the max-heap priority queue. The tree supports ordered reporting while the heap surfaces the highest-riskevent

Flow Chart

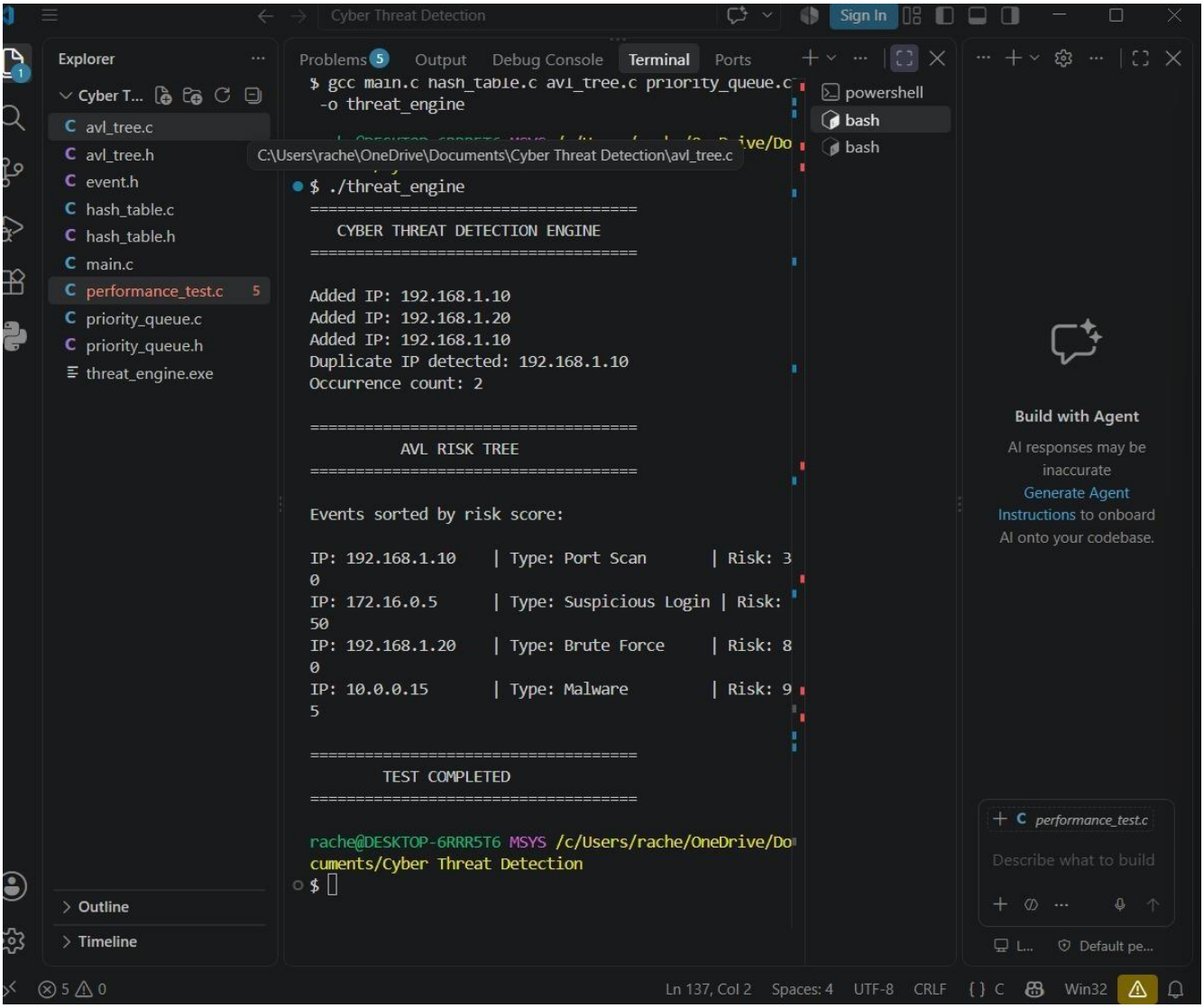
Cyber Threat Detection Engine



Pseudocode

```
START
Create hash table with selected bucket count
Create empty AVL tree
Create empty max-heap

FOR each security event
    Compute hash value from source IP
    Search hash chain
    IF IP exists
        Increment duplicate/occurrence count
        Mark event as repeated IP
    ELSE
        Insert IP into hash chain
    ENDIF

    Insert event into AVL tree using (risk, event ID)
    Insert event into max-heap using risk as priority
END FOR

Display duplicate IP information
Display AVL tree in descending risk order
Display highest-risk events from priority queue

FOR each performance configuration
    Generate test keys using selected distribution
    Measure hash insertion and search time
    Calculate load factor
    Record results
END FOR

Compare results and identify suitable configuration
END
```

Output:


Cyber Threat Detection
Sign In
Explorer
Cyber T...
avl_tree.c
avl_tree.h
event.h
hash_table.c
hash_table.h
main.c
performance_test.c 5
priority_queue.c
priority_queue.h
threat_engine.exe
Outline
Timeline
Problems 5
Output
Debug Console
Terminal
Ports
$ gcc main.c hash_table.c avl_tree.c priority_queue.c
-o threat_engine
C:\Users\rache\OneDrive\Documents\Cyber Threat Detection\avl_tree.c
$ ./threat_engine
CYBER THREAT DETECTION ENGINE
Added IP: 192.168.1.10
Added IP: 192.168.1.20
Added IP: 192.168.1.10
Duplicate IP detected: 192.168.1.10
Occurrence count: 2
AVL RISK TREE
Events sorted by risk score:
IP: 192.168.1.10 | Type: Port Scan | Risk: 30
IP: 172.16.0.5 | Type: Suspicious Login | Risk: 50
IP: 192.168.1.20 | Type: Brute Force | Risk: 80
IP: 10.0.0.15 | Type: Malware | Risk: 95
TEST COMPLETED
rache@DESKTOP-6RRR5T6 MSYS /c/Users/rache/OneDrive/Documents/Cyber Threat Detection
$
powershell
bash
bash
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
performance_test.c
Describe what to build
Default pe...
Ln 137, Col 2
Spaces: 4
UTF-8
CRLF
C
Win32

program

```
#include  <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define HASH_SIZE 17
#define MAX_EVENTS 100
#define MAX_HEAP 100

typedef struct Event {
    int id;
    char ip[16];
    char timestamp[20];
    char type[30];
    int risk;
} Event;

typedef struct HashNode {
    char ip[16];
    int count;
    struct HashNode *next;
} HashNode;

typedef struct AVLNode {
    Event event;
    int height;
    struct AVLNode *left, *right;
} AVLNode;

typedef struct {
    Event data[MAX_HEAP];
    int size;
} MaxHeap;

HashNode *hashTable[HASH_SIZE];

unsigned long hashFunction(const char *ip) {
    unsigned long hash = 5381;
    int c;
    while ((c = *ip++))
        hash = ((hash << 5) + hash) + c;
    return hash % HASH_SIZE;
}
```

```
void initHashTable(void) {
    int i;
    for (i = 0; i < HASH_SIZE; i++)
        hashTable[i] = NULL;
}

int insertOrUpdateIP(const char *ip) {
    unsigned long index = hashFunction(ip);
    HashNode *p = hashTable[index];

    while (p != NULL) {
        if (strcmp(p->ip, ip) == 0) {
            p->count++;
            return p->count;
        }
        p = p->next;
    }

    p = (HashNode *)malloc(sizeof(HashNode));
    strcpy(p->ip, ip);
    p->count = 1;
    p->next = hashTable[index];
    hashTable[index] = p;
    return 1;
}

int getHeight(AVLNode *n) {
    return n ? n->height : 0;
}

int maxInt(int a, int b) {
    return (a > b) ? a : b;
}

int compareEvent(Event a, Event b) {
    if (a.risk != b.risk)
        return a.risk - b.risk;
    return a.id - b.id;
}

AVLNode *newAVLNode(Event e) {
    AVLNode *node = (AVLNode *)malloc(sizeof(AVLNode));
```

```
    node->event = e;
    node->height = 1;
    node->left = node->right = NULL;
    return node;
}

AVLNode *rightRotate(AVLNode *y) {
    AVLNode *x = y->left;
    AVLNode *t = x->right;

    x->right = y;
    y->left = t;

    y->height = 1 + maxInt(getHeight(y->left), getHeight(y->right));
    x->height = 1 + maxInt(getHeight(x->left), getHeight(x->right));

    return x;
}

AVLNode *leftRotate(AVLNode *x) {
    AVLNode *y = x->right;
    AVLNode *t = y->left;

    y->left = x;
    x->right = t;

    x->height = 1 + maxInt(getHeight(x->left), getHeight(x->right));
    y->height = 1 + maxInt(getHeight(y->left), getHeight(y->right));

    return y;
}

int getBalance(AVLNode *n) {
    return n ? getHeight(n->left) - getHeight(n->right) : 0;
}

AVLNode *insertAVL(AVLNode *node, Event e) {
    int balance;

    if (node == NULL)
        return newAVLNode(e);

    if (compareEvent(e, node->event) < 0)
```

```
        node->left = insertAVL(node->left, e);
    else
        node->right = insertAVL(node->right, e);

    node->height = 1 + maxInt(getHeight(node->left), getHeight(node->right));
    balance = getBalance(node);

    if (balance > 1 && compareEvent(e, node->left->event) < 0)
        return rightRotate(node);

    if (balance < -1 && compareEvent(e, node->right->event) > 0)
        return leftRotate(node);

    if (balance > 1 && compareEvent(e, node->left->event) > 0) {
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }

    if (balance < -1 && compareEvent(e, node->right->event) < 0) {
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

void displayDescending(AVLNode *root) {
    if (root == NULL)
        return;

    displayDescending(root->right);
    printf("ID:%d IP:%s Type:%s Risk:%d Time:%s\n",
           root->event.id, root->event.ip, root->event.type,
           root->event.risk, root->event.timestamp);
    displayDescending(root->left);
}

void heapSwap(Event *a, Event *b) {
    Event temp = *a;
    *a = *b;
    *b = temp;
}
```

```
void heapInsert(MaxHeap *h, Event e) {
    int i, parent;
    h->data[h->size] = e;
    i = h->size;
    h->size++;

    while (i > 0) {
        parent = (i - 1) / 2;
        if (h->data[parent].risk >= h->data[i].risk)
            break;
        heapSwap(&h->data[parent], &h->data[i]);
        i = parent;
    }
}

Event heapDeleteMax(MaxHeap *h) {
    Event result = h->data[0];
    Event last = h->data[h->size - 1];
    int i = 0, left, right, largest;

    h->size--;

    if (h->size == 0)
        return result;

    h->data[0] = last;

    while (1) {
        left = 2 * i + 1;
        right = 2 * i + 2;
        largest = i;

        if (left < h->size &&
            h->data[left].risk > h->data[largest].risk)
            largest = left;

        if (right < h->size &&
            h->data[right].risk > h->data[largest].risk)
            largest = right;

        if (largest == i)
            break;
```

```
        heapSwap(&h->data[i], &h->data[largest]);
        i = largest;
    }

    return result;
}

void displayHashTable(void) {
    int i;
    HashNode *p;

    printf("\nDuplicate / Repeated IP Information:\n");
    for (i = 0; i < HASH_SIZE; i++) {
        p = hashTable[i];
        while (p != NULL) {
            if (p->count > 1)
                printf("%s -> %d occurrences\n", p->ip, p->count);
            p = p->next;
        }
    }
}

void freeHashTable(void) {
    int i;
    HashNode *p, *temp;

    for (i = 0; i < HASH_SIZE; i++) {
        p = hashTable[i];
        while (p != NULL) {
            temp = p;
            p = p->next;
            free(temp);
        }
        hashTable[i] = NULL;
    }
}

void freeAVL(AVLNode *root) {
    if (root == NULL)
        return;
    freeAVL(root->left);
    freeAVL(root->right);
    free(root);
```

```
}

void performanceTest(void) {
    int sizes[] = {17, 29, 53};
    int n = 50;
    int i, j, index;
    char key[16];
    clock_t start, end;
    double seconds;

    printf("\nPerformance Test - Hash Insertion\n");
    printf("Events: %d\n", n);
    printf("%-12s %-15s %-15s\n", "Buckets", "Load Factor", "Time(s)");

    for (j = 0; j < 3; j++) {
        int buckets = sizes[j];
        HashNode **table = (HashNode **)calloc(buckets, sizeof(HashNode *));
        start = clock();

        for (i = 0; i < n; i++) {
            sprintf(key, "10.0.%d.%d", (i / 250) % 250, i % 250);
            unsigned long h = 5381;
            const char *q = key;
            while ((index = *q++))
                h = ((h << 5) + h) + index;
            index = h % buckets;

            HashNode *p = (HashNode *)malloc(sizeof(HashNode));
            strcpy(p->ip, key);
            p->count = 1;
            p->next = table[index];
            table[index] = p;
        }

        end = clock();
        seconds = (double)(end - start) / CLOCKS_PER_SEC;
        printf("%-12d %-15.3f %-15f\n", buckets,
               (double)n / buckets, seconds);

        for (i = 0; i < buckets; i++) {
            HashNode *p = table[i], *temp;
            while (p) {
                temp = p;
```

```
            p = p->next;
            free(temp);
        }
    }
    free(table);
  }
}

int main(void) {
  Event events[] = {
    {1, "192.168.1.10", "10:01:12", "LOGIN_FAILED", 85},
    {2, "10.0.0.5",     "10:02:10", "PORT_SCAN",    95},
    {3, "192.168.1.10", "10:03:45", "LOGIN_FAILED", 90},
    {4, "172.16.0.8",   "10:04:20", "ACCESS",       40},
    {5, "10.0.0.5",     "10:05:33", "PORT_SCAN",    98},
    {6, "192.168.1.25", "10:06:11", "LOGIN_OK",     20},
    {7, "10.0.0.5",     "10:07:55", "BRUTE_FORCE", 100},
    {8, "172.16.0.8",   "10:08:19", "ACCESS",       35}
  };

  int n = sizeof(events) / sizeof(events[0]);
  int i, occurrences;
  AVLNode *root = NULL;
  MaxHeap heap;
  heap.size = 0;

  initHashTable();

  printf("REAL-TIME CYBER THREAT DETECTION ENGINE\n");
  printf("-----------------------------------------------\n");

  for (i = 0; i < n; i++) {
    occurrences = insertOrUpdateIP(events[i].ip);

    if (occurrences > 1)
      printf("Repeated IP detected: %s (Occurrence %d)\n",
          events[i].ip, occurrences);

    root = insertAVL(root, events[i]);
    heapInsert(&heap, events[i]);
  }

  displayHashTable();
```

```
    printf("\nEvents Ordered by Risk (AVL Tree):\n");
    displayDescending(root);

    printf("\nHighest-Risk Events (Priority Queue):\n");
    for (i = 0; i < 5 && heap.size > 0; i++) {
        Event e = heapDeleteMax(&heap);
        printf("ID:%d IP:%s Risk:%d Type:%s\n",
               e.id, e.ip, e.risk, e.type);
    }

    performanceTest();

    freeAVL(root);
    freeHashTable();

    return 0;
}
```

Input Used

The program uses the following sample events directly inside the C program:

ID | Source IP | Timestamp | Event Type | Risk
1 | 192.168.1.10 | 10:01:12 | LOGIN_FAILED | 85
2 | 10.0.0.5 | 10:02:10 | PORT_SCAN | 95
3 | 192.168.1.10 | 10:03:45 | LOGIN_FAILED | 90
4 | 172.16.0.8 | 10:04:20 | ACCESS | 40
5 | 10.0.0.5 | 10:05:33 | PORT_SCAN | 98
6 | 192.168.1.25 | 10:06:11 | LOGIN_OK | 20
7 | 10.0.0.5 | 10:07:55 | BRUTE_FORCE | 100
8 | 172.16.0.8 | 10:08:19 | ACCESS | 35

4. Use of Modern Tools

The implementation is designed for a standard C compiler such as GCC and can be developed and tested using an IDE or command-line environment. Git can be used for version control and GitHub can be used for the required upload. The C standard library functions clock(), malloc(), calloc(), free(), strcmp(), strcpy(), and sprintf() support timing, dynamic memory, string handling, and test-data generation.

For evidence, the student can capture the compiler output showing repeated-IP detection, AVL risk ordering, priority-queue results, and performance-test measurements.

5.Results and Validation

Event / Check	Expected Observation
Duplicate IP detection	Repeated source IPs are reported with occurrence counts.
AVL tree ordering	Events appear from highest risk to lowest risk.
Priority queue	Risk 100 is surfaced before risk 98, 95, 90 and 85.
Hash performance	Lower load factors generally reduce expected chain length.
Skewed distribution	Repeated keys exercise duplicate detection and longer chains.
Memory management	Allocated hash and AVL nodes are released before termination.

Illustrative expected risk ordering for the supplied input is: ID 7 (100), ID 5 (98), ID 2 (95), ID 3 (90), ID 1 (85), ID 4 (40), ID 8 (35), ID 6 (20). Exact timing values depend on the computer, compiler, operating system, and system load.

6.Analysis and Engineering Decision

Separate chaining was selected instead of open addressing because it provides straightforward collision handling and remains flexible when the number of repeated or colliding IP addresses changes. It also avoids the need for tombstone management during deletion, although this demonstration does not require deletion.

The AVL tree was selected because the assignment requires a self-balancing ordered structure. Compared with an ordinary binary search tree, AVL balancing prevents highly skewed tree shapes and maintains O(log n) height. The priority queue is complementary: the AVL tree is appropriate for ordered reporting, while the heap is optimized for repeatedly accessing the highest-risk event.

A lower hash-table load factor generally provides shorter chains and faster expected lookup, at the cost of more memory. A higher load factor uses memory more efficiently but can increase collisions. Under skewed distributions, a good hash function and appropriate table size become especially important.

7.Broader Considerations

Sustainability: Efficient data structures reduce unnecessary computation and memory usage when large security logs are processed, supporting scalable infrastructure.

Society and safety: Faster detection of suspicious activity can help organizations respond to attacks such as brute-force login attempts and port scanning.

Ethics and privacy: Real-world deployments should minimize collection of personal information, protect security logs, apply access controls, and follow applicable organizational and legal requirements.

Professional responsibility: The implementation should be tested carefully because incorrect duplicate detection or risk ranking could cause analysts to miss important security events.

8. Conclusion

The proposed real-time cyber threat detection engine combines three suitable data structures: a hash table for fast IP lookup and duplicate detection, an AVL tree for dynamically ordered risk storage, and a max-heap priority queue for highest-risk event retrieval. The integrated C implementation demonstrates how different abstract data types can cooperate in a high-volume security-log processing system.

The solution satisfies the major functional requirements of the assignment and provides a performance-testing framework for different load factors. Limitations include the fixed demonstration capacity, simplified risk model, and use of synthetic sample data. Future improvements could include dynamic resizing, open-addressing comparison, Red-Black Tree implementation, persistent storage, multithreaded ingestion, and more realistic benchmark datasets.

9. Student Reflection

This assignment helped me understand that choosing a data structure depends on the operation that must be optimized. I learned how hashing can provide fast duplicate detection, how AVL rotations maintain balance, and how a priority queue can continuously identify the most important event.

I also learned that theoretical complexity alone is not enough for a practical system. Load factor, key distribution, memory usage, and collision behavior can affect real performance. If additional time and resources were available, I would add dynamic hash-table resizing, compare chaining with open addressing, implement a Red-Black Tree alternative, and perform larger benchmark experiments.

10. References

- Assignment document: CSA0309 – Data Structures – Slot B, “Build a Real-Time Cyber Threat Detection Engine Using Hashing and Self-Balancing Trees.”
- Assignment Format – Common Course Assignment Format, provided for report structure.
- C standard library documentation for stdio.h, stdlib.h, string.h, and time.h.
- Course notes and classroom material on hashing, AVL trees, collision resolution, and priority queues.

F. Common Assessment Rubric

Assessment Criterion	**Marks**
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO2	PO1, PO2	L3/L4	10
Application of knowledge	CO2, CO3	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO3, CO4	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO4, CO5	PO5	L3/L4	10
Validation	CO5	PO2, PO4	L4/L5	15
Analysis & justification	CO3, CO5	PO2, PO3	L4/L5	15
Broader considerations	CO5	PO6, PO7	L4/L5	5
Documentation & reflection	CO2, CO5	PO10	L3/L4	5

Note: The PO mapping above is a proposed student-report mapping based on the assignment requirements. Faculty may modify the PO mapping to match the officially assessed programme outcomes.

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis – Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____